

Chapter 1 Introduction

1.1 Research Background and O-RAN Architecture Evolution

In the architectural evolution of the fifth-generation (5G) and the upcoming sixth-generation (6G) mobile communication standards, the Radio Access Network (RAN) is undergoing a fundamental transformation from the traditional monolithic base station to a highly virtualized, cloudified, and disaggregated architecture. Under the traditional architecture, baseband processing and Radio Frequency (RF) functions were tightly bound to the special hardware equipment of a single vendor, which not only limited the flexibility of network deployment but also significantly increased the capital and operational expenditures of operators.

With the rise of Cloud-RAN (C-RAN), disaggregated deployment methods have gradually become popular, centralizing the Baseband Processing Unit (BBU) into a remote BBU Pool while leaving the Remote Radio Head (RRH) at the edge site, connecting the two via the Common Public Radio Interface (CPRI). However, the C-RAN architecture is still limited by the massive fronthaul bandwidth requirements and strict latency constraints. To overcome these bottlenecks and completely break the dilemma of vendor lock-in, the O-RAN Alliance [?] and the 3rd Generation Partnership Project (3GPP) jointly promoted further disaggregation of the RAN, formally splitting the traditional BBU and RRH logically and physically into three independent network entities: the Central Unit (CU), Distributed Unit

(DU), and Radio Unit (RU) [?,?].

In this disaggregated model, functional split options have become the most critical design core for determining network performance, hardware complexity, bandwidth consumption, and latency tolerance [?,?]. 3GPP has standardized a variety of functional split options to adapt to diverse network requirements and deployment scenarios.

1.1.1 Split Option 2

Split Option 2 is a High Layer Split (HLS) architecture that primarily separates the CU and DU. Under this split, higher-layer protocols such as the Packet Data Convergence Protocol (PDCP) and Radio Resource Control (RRC) run centrally in the CU, while lower-layer Radio Link Control (RLC), Medium Access Control (MAC), and Physical Layer (PHY) remain in the DU. This architecture has relatively loose fronthaul bandwidth and latency requirements, making it easy to deploy over general packet-switched networks. The design of its synchronization mechanism is also very simple; due to its high tolerance, it only relies on flow control and deep buffering to operate stably [?], and is widely recognized as the standard configuration with the best cost-effectiveness ratio, especially suitable for non-real-time data transmission scenarios.

1.1.2 Split Option 6

Split Option 6, also known as the MAC-PHY split, precisely places the architectural separation point between the MAC layer and the PHY layer.

In this option, the DU is responsible for processing upper-layer protocols including the MAC layer, while the RU retains complete physical layer (PHY) processing capabilities. Since the fronthaul network transmits MAC Protocol Data Units (MAC PDUs) scheduled by the MAC layer, its bandwidth requirements are dynamically adjusted based on the actual load of the users. Notably, although the nFAPI-based Split Option 6 architecture also supports deployment over general packet-switched networks, its synchronization requirements are completely different from Split 2. It must strictly follow the scheduler's synchronization baseline at every slot level, ensuring data is delivered precisely on time from the VNF to the PNF. During this process, the operating system scheduling delays on the VNF side, as well as the unknown transmission delays and jitter introduced by the packet-switched network, are major challenges that must be overcome to maintain precise time synchronization [?].

1.1.3 Split Option 7.2

Split Option 7.2x (Low-PHY split) places the split point inside the physical layer, and is currently the primary Lower Layer Split (LLS) standard promoted by the O-RAN Alliance [?]. Although moving partial computations (such as FFT/IFFT) to the RU alleviates fronthaul pressure, its fronthaul transmits frequency-domain IQ symbols processed by the physical layer, demanding extremely high bandwidth. Because of this, Split 7.2 must be deployed under very strict network conditions, and its reliance on hardware equipment (such as hardware-level Precision Time Protocol, PTP) results in extremely high deployment costs [?]. However, precisely

because it has a strict private network environment and transmits a stable, continuous stream of IQ data, its synchronization mechanism is relatively stable and does not require special handling for latency variation issues caused by different offered loads.

1.1.4 Split Option 8

Split Option 8 is the most traditional lower-layer split architecture, placing the split point between the Radio Frequency (RF) and the Physical Layer (PHY). Under this architecture, all baseband processing functions (including complete PHY, MAC, and higher-layer protocols) are centralized in the DU or CU, while the RU is only responsible for RF signal transmission and reception. This configuration produces a fronthaul data stream dominated by time-domain IQ samples, usually relying on CPRI or enhanced CPRI (eCPRI) for transmission. Although it highly centralizes hardware resources, its fronthaul network bandwidth requirements are extremely high, and it is strictly constrained by system bandwidth and antenna count. Furthermore, its latency limits are extremely strict (maximum allowable one-way latency is $250\mu s$ [?]), thus, similar to Split 7.2, it heavily relies on high-quality private optical networks and precise timing synchronization hardware.

1.2 Network Architecture and Challenges Based on Split Option 6

This thesis focuses on the architectural design and application of Split Option 6. As mentioned earlier, while adopting lower-layer splits (such as Split Option 7.2) can further centralize computational resources, it requires a more expensive PTP deployment environment and a transmission network with extremely high demands for low latency, which will lead to high capital and operational expenditures. In light of this, choosing the Split Option 6 approach, which leaves the complete physical layer (Monolithic PHY) at the remote node, can strike the best balance between resource requirements and cost-effectiveness. According to research by Khan et al. [?], cost-benefit trade-off analysis for 5G NR small cells indicates that compared to strict lower-layer splits, the architecture separating the MAC and PHY layers can significantly reduce the required fronthaul bandwidth while ensuring goodput, thereby achieving the goal of lowering overall deployment costs. Therefore, in non-ideal or resource-constrained network environments, Split Option 6 becomes the ideal choice that balances performance and cost.

1.2.1 Introduction to the FAPI Interface

Within the internal architecture of 5G base stations, communication between the MAC layer and PHY layer relies on standardized interfaces. The Functional Application Platform Interface (FAPI) introduced by the Small Cell Forum (SCF) was created for this purpose [?]. FAPI provides a stan-

standardized format for control messages and data payloads, allowing MAC and PHY components from different vendors to interoperate. However, the traditional FAPI design assumes that the MAC and PHY are located within the same physical device, and the two exchange data quickly with low latency via shared memory, which limits the geographical disaggregation and distributed deployment of base station functions.

1.2.2 Introduction to the nFAPI Interface

To overcome the physical deployment limitations of FAPI and realize the Split Option 6 architecture across networks, SCF further developed the network Functional Application Platform Interface (nFAPI) [?]. The core innovation of nFAPI lies in encapsulating FAPI messages, which originally relied on shared memory, into a network packet protocol that can be transmitted over Ethernet or IP networks [?]. This technological breakthrough grants Virtual Network Functions (VNFs) immense flexibility, allowing cloudified MAC functions running on Commercial Off-The-Shelf (COTS) servers to remotely control physical PHY devices on Physical Network Functions (PNFs) over packet-switched networks.

It is particularly important to note that according to 3GPP TR 38.801 [?] specifications, the ideal fronthaul one-way latency limit for Split Option 6 is $250\mu s$. However, the SCF provides a more practical, special interpretation for nFAPI's latency tolerance in its specifications [?,?]. The SCF indicates that nFAPI is designed for packet-switched IP networks, and its fronthaul latency budget is primarily determined by the configuration of the autonomous HARQ retransmission mechanism. By configuring

two autonomous HARQ retransmissions, nFAPI can tolerate up to 6 ms of fronthaul delay (including frame delay and delay jitter) [?]. Furthermore, in a non-ideal fronthaul environment, by fine-tuning the HARQ parameters of the S-DU (such as K0, K1, K2) and the RACH timer, combined with the built-in delay management mechanism of the nFAPI P7 interface and the SCTP protocol adapted for the P5 interface, nFAPI can even support highly delayed and highly jittered transmission conditions of up to 30 ms [?]. This design specifically for non-ideal networks makes Split Option 6 highly practical and flexible in deployment.

1.2.3 Comprehensive Comparison of Split Technologies



To more intuitively present the differences between different split options, Table 1.1 provides a comprehensive comparison of Split 2, Split Option 6, Split 7.2x, and Split 8 in terms of protocol boundaries, bandwidth requirements, and latency limits.

This thesis focuses on the deployment and optimization of nFAPI because it provides excellent network flexibility, allowing network operators to logically split the base station and deploy it on two independent machines, even connected via general packet-switched networks. However, this physical split also brings immense challenges: in a non-deterministic network environment, how to maintain time synchronization between the two nodes and conduct precise delay management has become the primary difficulty in ensuring stable system operation.

Table 1.1: 5G RAN Split Deployment Comparison Table

Split Option	Split 2	Split 6 (nFAPI)	Split 7.2x	Split 8
Protocol Boundary	PDCP / RLC	MAC / PHY	Low-PHY / High-PHY	RF / PHY
Fronthaul Data Type	PDCP PDU	MAC PDU (Packetized)	Frequency-domain IQ symbols	Time-domain IQ samples
Standardized Interface	F1	SCF nFAPI	O-RAN WG4 eCPRI	CPRI / eCPRI
Bandwidth Driver	Actual user traffic	Actual user traffic	Bandwidth, spatial layers	System bandwidth, antennas
Latency Limit [?,?]	1.5 ~ 10ms	250μs (3GPP) 6 ~ 30ms (SCF)	250μs	250μs
Optimal Deployment	Non-real-time services	Small cells, indoor private networks	Macro cells	C-RAN, dedicated fiber

1.3 Research Motivation and Problem Definition

Although Split Option 6 has flexible timing adjustment capabilities and better tolerance for non-ideal fronthaul networks, it still faces severe challenges in practical deployment. As radio access networks move towards virtualization and cloudification, the hosted MAC layer functions are typically deployed as Virtual Network Functions (VNFs) on general-purpose processors. However, in this general-purpose computing environment, non-deterministic operating system scheduling jitter and processing delay uncertainty often impact timing-sensitive baseband processing [?,?].

Specifically, in existing open-source implementations (such as OpenAirInterface), the timing control of the nFAPI interface mostly relies on a statically configured slots ahead parameter. This hardcoded approach cannot dynamically adapt to the rapidly changing transmission delays and network jitter in packet-switched networks. When bursty congestion occurs in the fronthaul network or latency spikes appear in the VNF computing environment, scheduling commands and control messages are highly likely to miss their scheduled timing deadlines, thereby generating Missing Slot Packet errors. These types of errors not only lead to communication interruptions but also limit the VNF's elastic scalability in dynamic cloud resource environments.

Therefore, the core research question of this thesis is defined as follows: In a cloud environment with non-deterministic processing loads and network jitter, how to dynamically and accurately compensate for end-to-

end latency in the nFAPI Split Option 6 architecture to ensure the stability of timing synchronization?

1.4 Related Works

In recent years, the rapid development of the Open Radio Access Network (O-RAN) architecture has prompted many studies to optimize the performance and network latency management of the Split Architecture. For instance, the X5G testbed proposed by Villa et al. [?] demonstrated an advanced 5G O-RAN deployment solution based on hardware acceleration. This research successfully achieved commercial-grade peak throughput by offloading the massive computations of the Physical Layer (PHY) to dedicated NVIDIA GPUs and Mellanox SmartNICs. However, this architecture, which pursues ultimate performance, highly relies on expensive special hardware equipment, and its prerequisite is that it must operate in an ideal network environment equipped with hardware-level Precision Time Protocol (PTP) synchronization. These strict conditions greatly limit its applicability and deployment flexibility in general packet-switched networks filled with unpredictable latency and jitter.

To address the limitations of high hardware costs and strict network requirements mentioned above, this thesis proposes a highly universal software-defined solution for non-ideal fronthaul environments. Unlike X5G, which relies on hardware acceleration to overcome latency issues, and unlike traditional methods that rely on static configurations to compromise on delay jitter, this thesis designs an Adaptive Slots Ahead Control Mechanism based on Exponentially Weighted Moving Average (EWMA) delay man-

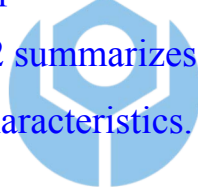
agement specifically for the nFAPI architecture. This mechanism can run on general COTS servers, dynamically tracking and filtering network jitter, and actively adapting to unpredictable network congestion. Experimental results show that the method proposed in this thesis can effectively stabilize the HARQ loop latency (RTT) and maintain system throughput without relying on high-end special hardware or precise clock synchronization equipment, providing a more cost-effective and flexible alternative for O-RAN deployment.

Simultaneously, current academic research on O-RAN delay management has explored various dimensions to address non-deterministic latency. At the intra-node level, Lee et al. [?] proposed an adaptive transmission strategy based on the FAPI interface, utilizing the variance in Layer 1 software (L1SW) processing times to opportunistically advance transmissions, which effectively complements end-to-end transport delay management. For strict deterministic latency requirements in ultra-reliable low-latency communication (uRLLC), Adamuz-Hinojosa et al. [?] introduced the ORANUS framework, employing Stochastic Network Calculus (SNC) to provide strict mathematical bounds for latency violations. Furthermore, from the perspective of User Equipment (UE) Service Quality (QoS), Lv et al. [?] developed UQ-vRAN, which emphasizes the joint optimization of MAC scheduling and higher-layer orchestration through precise traffic digital twins.

While these existing literatures [?,?,?,?] have proposed optimization schemes ranging from specific intra-node CPU adjustments to long-term QoS orchestration, they generally do not directly address the severe cross-node synchronization failures caused by non-deterministic fronthaul jitter in intermediate-layer splits. This thesis fills the critical gap by targeting the

microsecond-level phase synchronization and latency challenges inherent in the nFAPI Split Option 6 architecture. Specifically, we focus on the fact that the Small Cell Forum (SCF) 225 nFAPI specification [?] defines the standard Timing Info message and interface-level delay management fields, yet existing open-source implementations only support static parameter initialization. Our proposed dynamic timing control completes this standardized framework by implementing an adaptive timing closed-loop feedback, presenting a unique academic contribution to realizing dynamic time synchronization under a pure software-defined network architecture without relying on specialized hardware.

To clearly position the proposed work within the current state-of-the-art (SOTA) literature, Table 1.2 summarizes the comparison across key architectural and performance characteristics.



1.5 Thesis Contributions

In response to the aforementioned issues, the main contributions of this thesis are as follows:

- **Multi-dimensional Latency Quantitative Analysis for nFAPI Split Option 6:** We systematically analyzed and quantified the non-deterministic factors affecting synchronization failure, including Linux kernel scheduling jitter, VNF processing load, and network queuing delays.
- **Adaptive Slots Ahead Mechanism:** We proposed a dynamic adjustment mechanism based on Node Sync and Delay Management. It

Table 1.2: Comparison of Proposed Work with State-of-the-Art Solutions

Metric / Feature	X5G [?]	Lee <i>et al.</i> [?]	Proposed Work
Split Option	Split 7.2	Split 6 (FAPI)	Split 6 (nFAPI)
Frontend Type	Dedicated Fiber / TSN	Shared Memory (Intra-node)	Packet-switched Ethernet
Hardware Dependency	GPU / SmartNIC / PTP	COTS (No Acceleration)	COTS (No Acceleration)
Control Resolution	Symbol-level	OS / Process-level	Slot / TTI-level
Adaptation Domain	Hardware Offload	Intra-node CPU Scheduling	Cross-node Jitter & Delay

uses an EWMA model to smooth timing information to estimate real-time jitter trends, and dynamically adjusts the slots ahead (S_{ahead}) to ensure the punctuality of scheduling messages.

- **End-to-End Commercial Testbed Verification:** We implemented the proposed mechanism on the OpenAirInterface platform and combined it with a commercial UE and O-RU for verification. Experimental results show that this mechanism can effectively eliminate missing slot packet errors and maintain stable network throughput and low latency characteristics in highly congested environments.
- **Open-Source nFAPI Platform Contribution to Upstream OAI:** We submitted our adaptive timing control and nFAPI modifications as a merge request (MR) to the official OpenAirInterface (OAI) [?] repository. This contribution was merged into the official codebase. It provides a standard open-source nFAPI platform that follows the Small Cell Forum (SCF) specification for future development and testing.